

Feature Catalog

Repository-backed feature entries, flows, edge cases and status.



Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository:
rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @
3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Repository-backed feature entries, flows, edge cases and status. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

Phone OTP sign-in

Current implementation

Who uses it

Customer

Current status

Current implementation

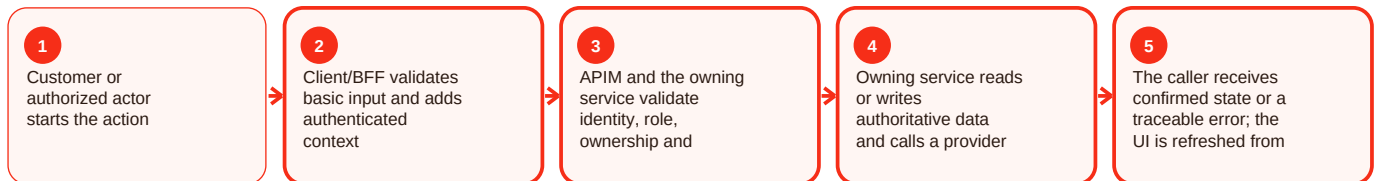
Summary

Phone otp sign-in is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For Phone OTP sign-in, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. What to do: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. What to do: Confirm intended access; do not bypass authorization.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md.

Customer profile

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

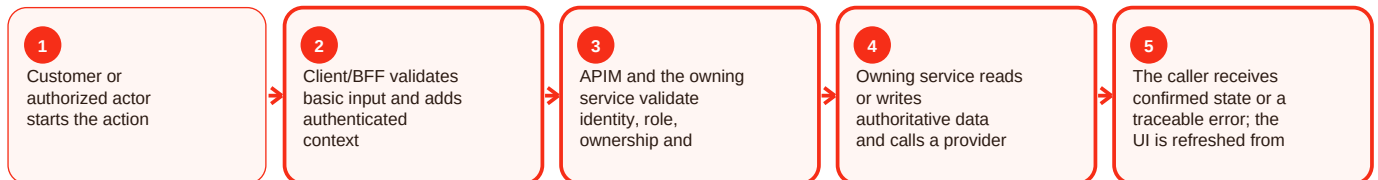
Summary

Customer profile is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Customer profile, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Discovery

Current implementation

Who uses it

Customer

Current status

Current implementation

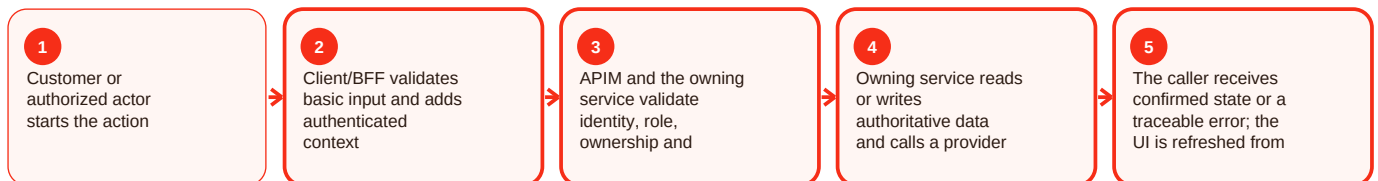
Summary

Discovery is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Discovery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Search

Current implementation

Who uses it

Customer

Current status

Current implementation

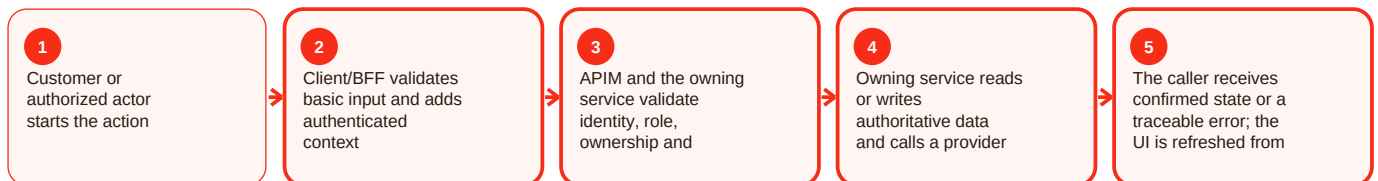
Summary

Search is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Search, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Meal detail

Current implementation

Who uses it

Customer

Current status

Current implementation

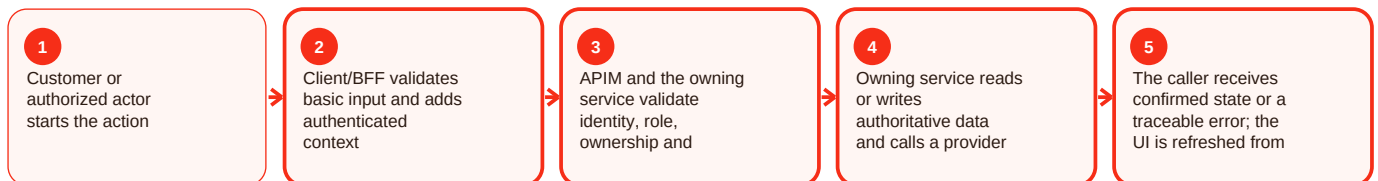
Summary

Meal detail is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Meal detail, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Favorites

Current implementation

Who uses it

Customer

Current status

Current implementation

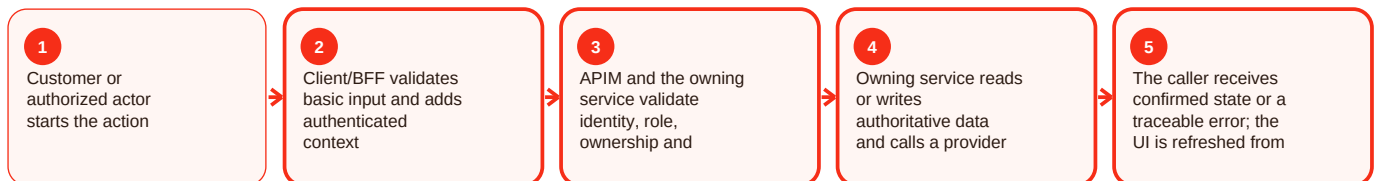
Summary

Favorites is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Favorites, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Ratings and reviews

Current implementation

Who uses it

Customer

Current status

Current implementation

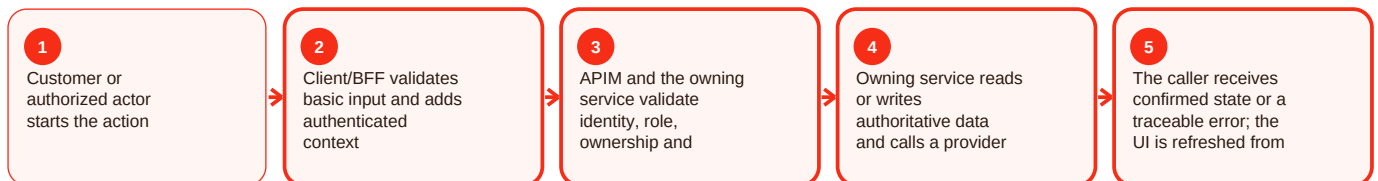
Summary

Ratings and reviews is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Ratings and reviews, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Cart

Current implementation

Who uses it

Customer

Current status

Current implementation

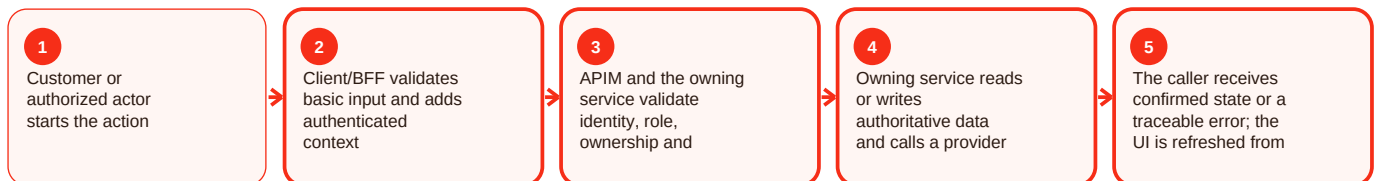
Summary

Cart is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For Cart, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Checkout rejected: Saved address, cart or catalog state is not valid. What to do: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. What to do: Show the backend quote; do not preserve a client-calculated amount.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md.

Quantity controls

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

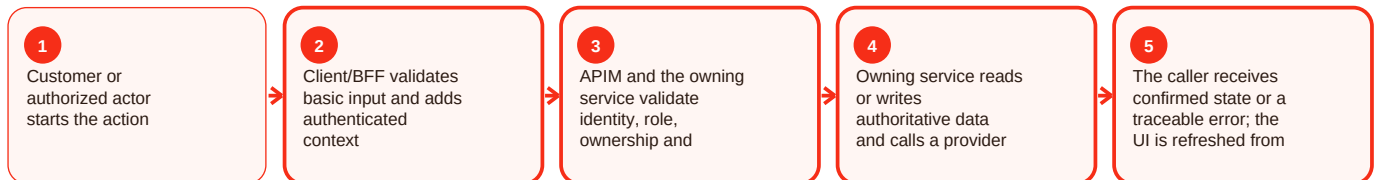
Summary

Quantity controls is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Quantity controls, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Authoritative checkout quote

Current implementation

Who uses it

Customer

Current status

Current implementation

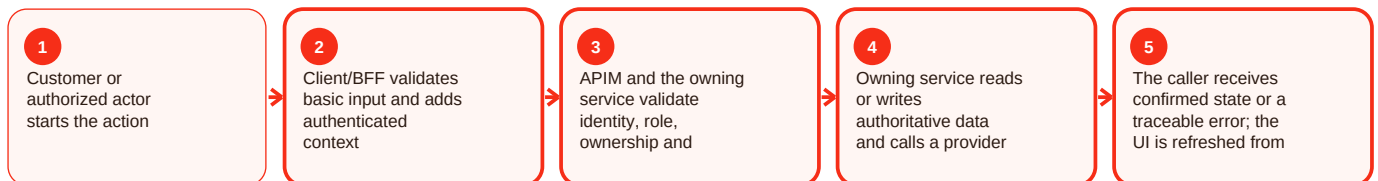
Summary

Authoritative checkout quote is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For Authoritative checkout quote, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Checkout rejected: Saved address, cart or catalog state is not valid. What to do: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. What to do: Show the backend quote; do not preserve a client-calculated amount.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md.

Addresses

Current implementation

Who uses it

Customer

Current status

Current implementation

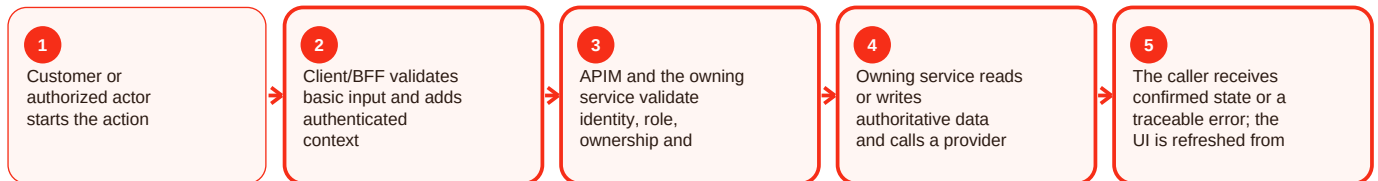
Summary

Addresses is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For Addresses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Current location unavailable: Permission/device/geocoder failed. What to do: Allow manual saved-address entry and keep checkout dependent on a valid saved address.
Address rejected at checkout: Address is missing ownership/required fields/coordinates. What to do: Fix the saved address in User-Chef before placing the order.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md.

Current location and maps

Current implementation

Who uses it

Customer

Current status

Current implementation

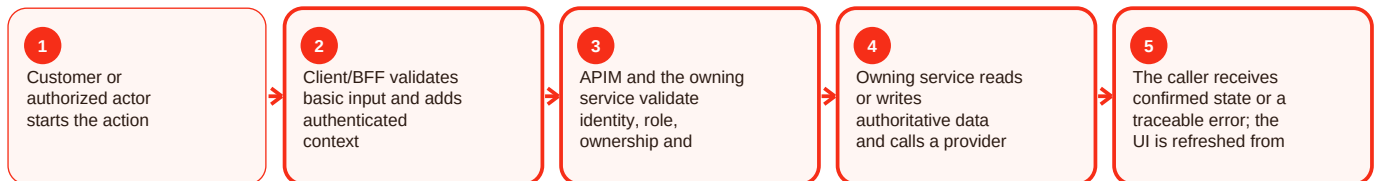
Summary

Current location and maps is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For Current location and maps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Current location unavailable: Permission/device/geocoder failed. What to do: Allow manual saved-address entry and keep checkout dependent on a valid saved address.
Address rejected at checkout: Address is missing ownership/required fields/coordinates. What to do: Fix the saved address in User-Chef before placing the order.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md.

Cashfree hosted payment

Current implementation

Who uses it

Customer

Current status

Current implementation

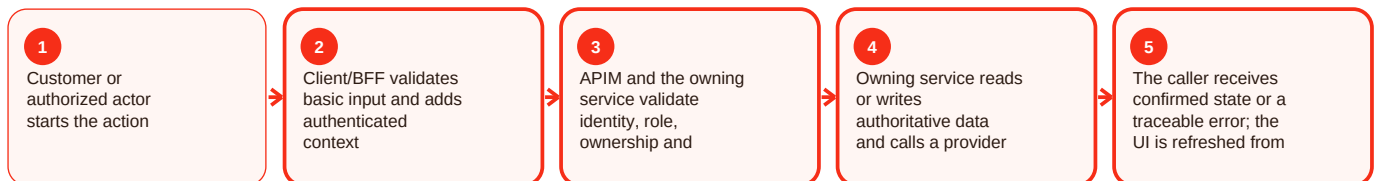
Summary

Cashfree hosted payment is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree hosted payment, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Payment result is unclear: Provider status/callback is delayed or ambiguous. What to do: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. What to do: Verify the raw body with the server secret; never disable signature checks.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md.

Razorpay callback acceptance

Current implementation

Who uses it

Customer

Current status

Current implementation

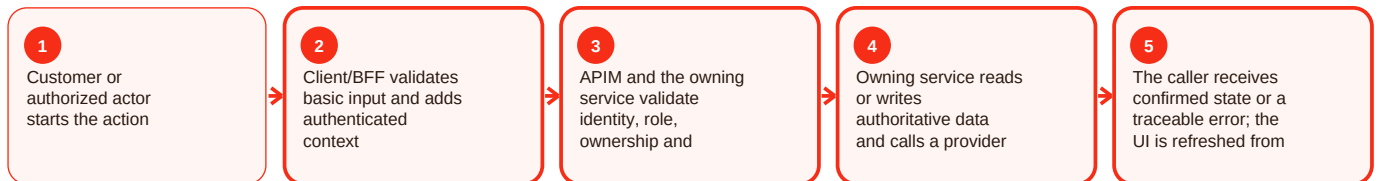
Summary

Razorpay callback acceptance is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay callback acceptance, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Payment result is unclear: Provider status/callback is delayed or ambiguous. What to do: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. What to do: Verify the raw body with the server secret; never disable signature checks.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md.

Order creation

Current implementation

Who uses it

Customer

Current status

Current implementation

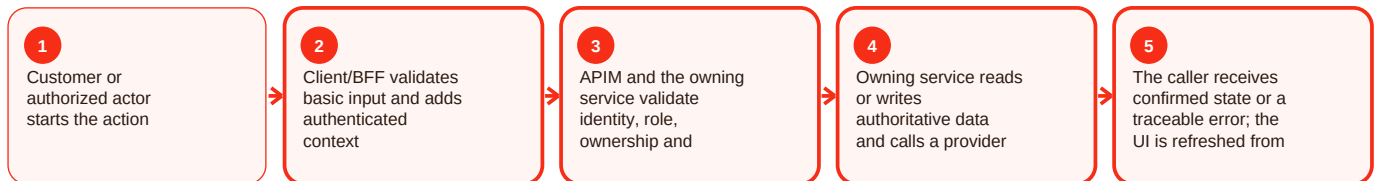
Summary

Order creation is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Order creation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Order confirmation

Current implementation

Who uses it

Customer

Current status

Current implementation

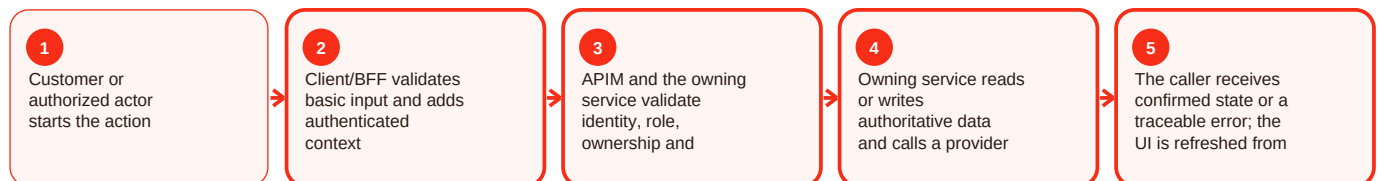
Summary

Order confirmation is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Order confirmation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Order history

Current implementation

Who uses it

Customer

Current status

Current implementation

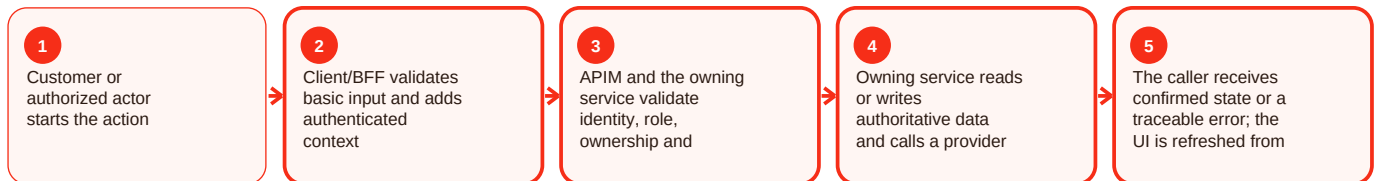
Summary

Order history is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Order history, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Order tracking

Current implementation

Who uses it

Customer

Current status

Current implementation

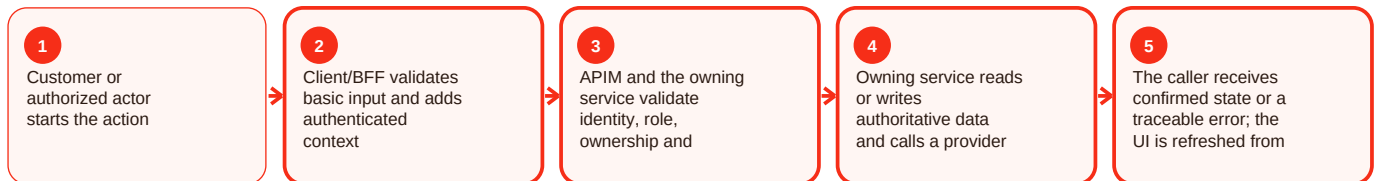
Summary

Order tracking is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Order tracking, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Cancellation

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

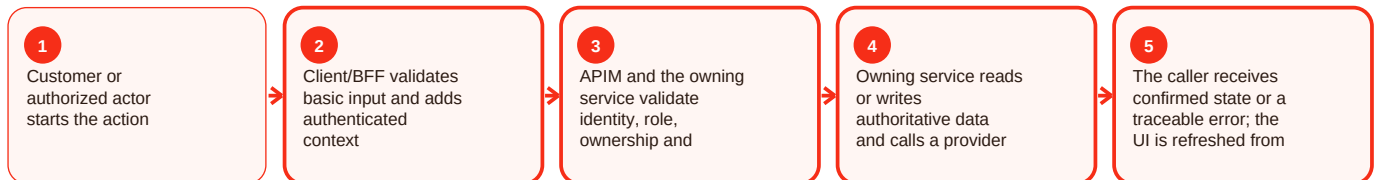
Summary

Cancellation is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Cancellation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Refund

Current implementation

Who uses it

Customer

Current status

Current implementation

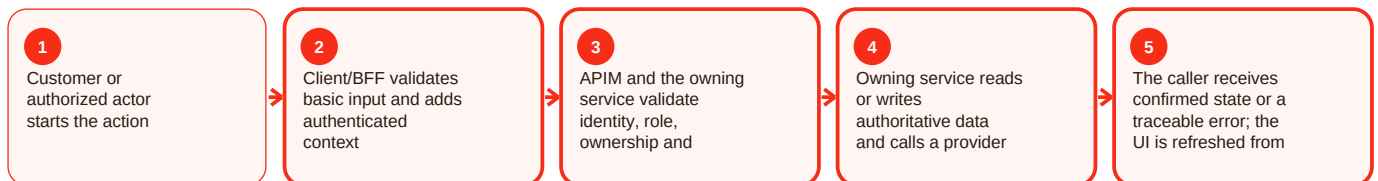
Summary

Refund is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Refund, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Payment result is unclear: Provider status/callback is delayed or ambiguous. What to do: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. What to do: Verify the raw body with the server secret; never disable signature checks.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md.

Delivery ETA

Current implementation

Who uses it

Customer

Current status

Current implementation

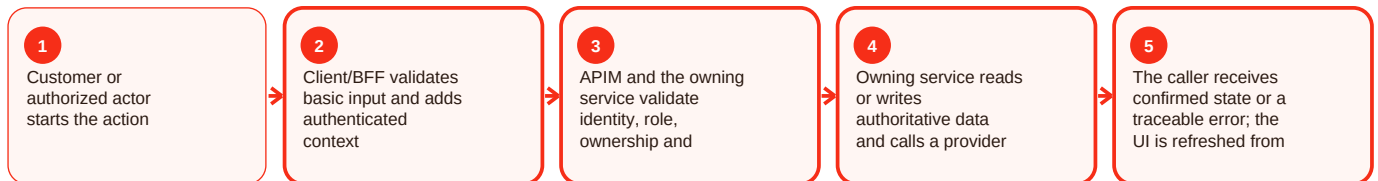
Summary

Delivery eta is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Delivery ETA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Shortage handling

Current implementation

Who uses it

Customer

Current status

Current implementation

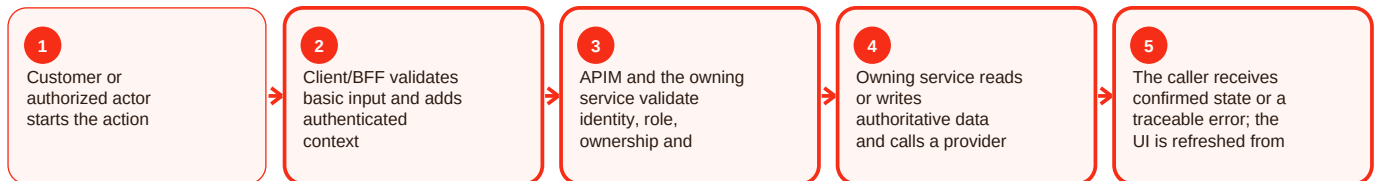
Summary

Shortage handling is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Shortage handling, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Completion proof

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

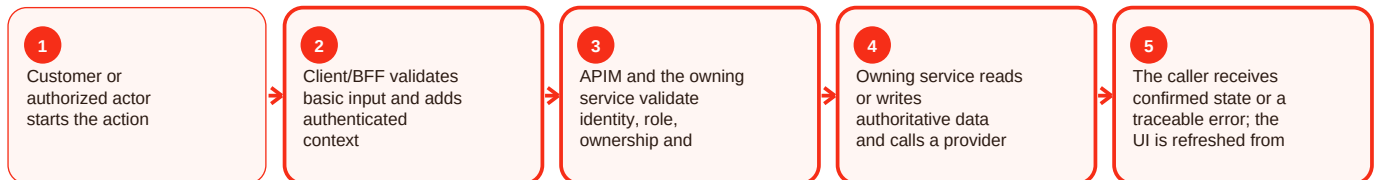
Summary

Completion proof is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Completion proof, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Plans

Current implementation

Who uses it

Customer

Current status

Current implementation

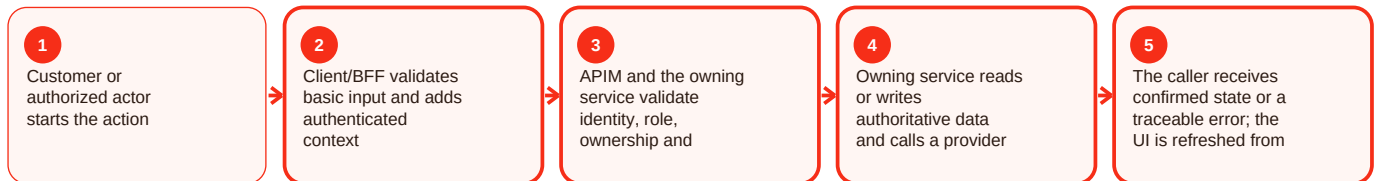
Summary

Plans is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For Plans, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Lifecycle action rejected: Current subscription state does not allow it. What to do: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. What to do: Document the decision before implementing it.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/subscription-service/README.md`; `apps/customer-web-next/src/components/plans/`; `apps/customer-web-next/src/components/subscriptions/`; commit 3225f7fa.

Subscriptions

Current implementation

Who uses it

Customer

Current status

Current implementation

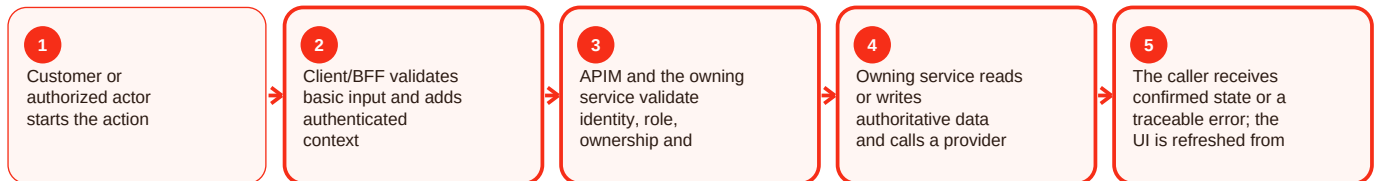
Summary

Subscriptions is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For Subscriptions, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Lifecycle action rejected: Current subscription state does not allow it. What to do: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. What to do: Document the decision before implementing it.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa.

Entitlements and gated services

Current implementation

Who uses it

Customer

Current status

Current implementation

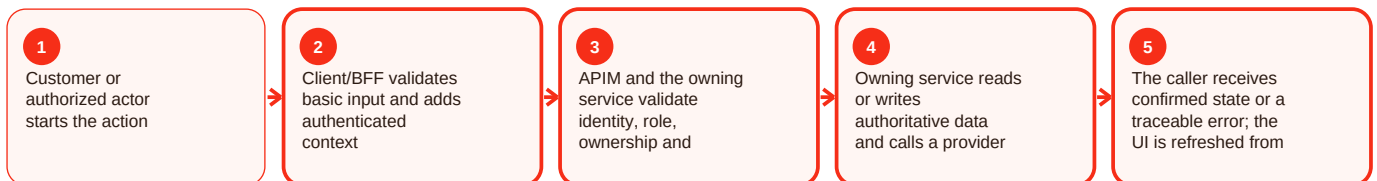
Summary

Entitlements and gated services is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For Entitlements and gated services, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Lifecycle action rejected: Current subscription state does not allow it. What to do: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. What to do: Document the decision before implementing it.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa.

Notification center

Current implementation

Who uses it

Customer

Current status

Current implementation

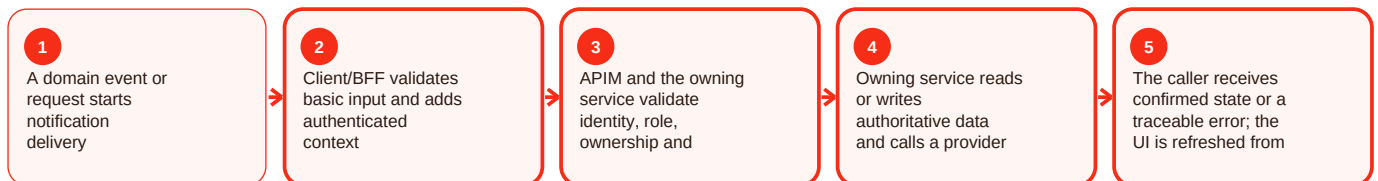
Summary

Notification center is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For Notification center, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

External channel stays pending: Provider adapter is disabled or unavailable. What to do: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. What to do: Use idempotency evidence before resending.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/.

Notification preferences

Current implementation

Who uses it

Customer

Current status

Current implementation

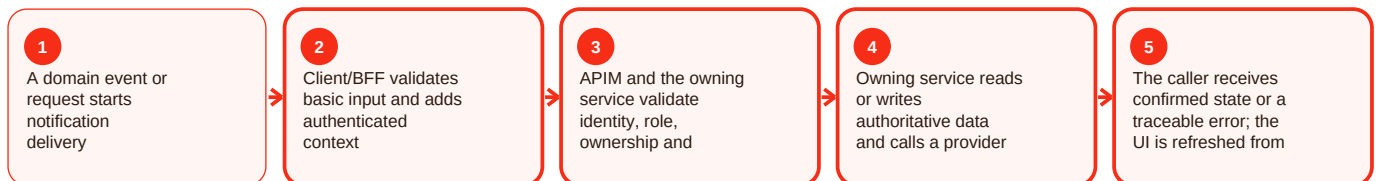
Summary

Notification preferences is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For Notification preferences, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

External channel stays pending: Provider adapter is disabled or unavailable. What to do: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. What to do: Use idempotency evidence before resending.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/notification-service/README.md`; `apps/customer-web-next/src/components/notifications/`.

Push device tokens

Current implementation

Who uses it

Customer

Current status

Current implementation

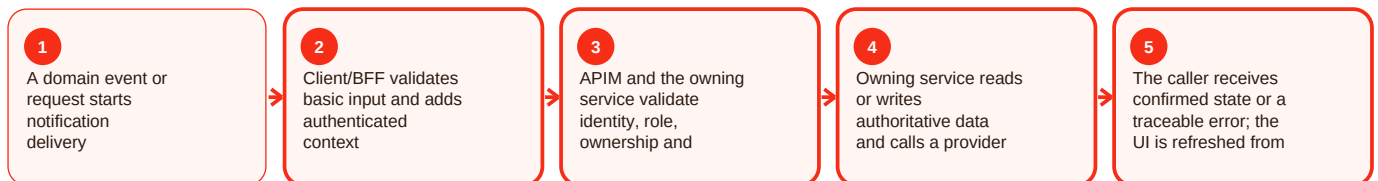
Summary

Push device tokens is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For Push device tokens, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

External channel stays pending: Provider adapter is disabled or unavailable. What to do: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. What to do: Use idempotency evidence before resending.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/.

Chef onboarding

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

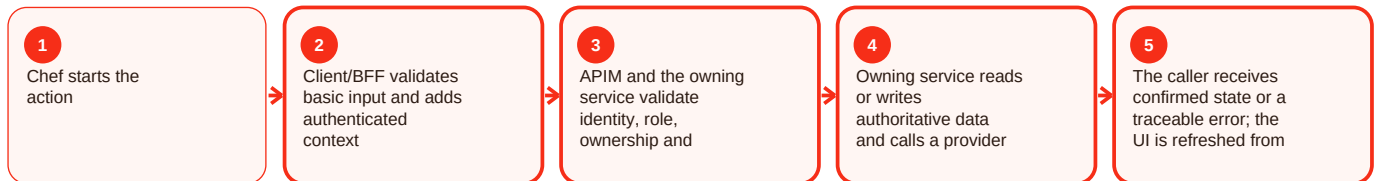
Summary

Chef onboarding is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Chef onboarding, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/user-chef-service/README.md`; `apps/customer-web-next/src/components/chef/`; `apps/customer-web-next/src/components/chef-mode/`.

Chef verification

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

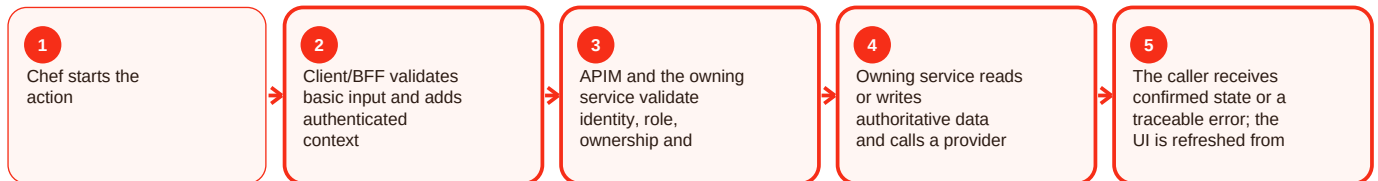
Summary

Chef verification is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Chef verification, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/.

Customer/Chef mode

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

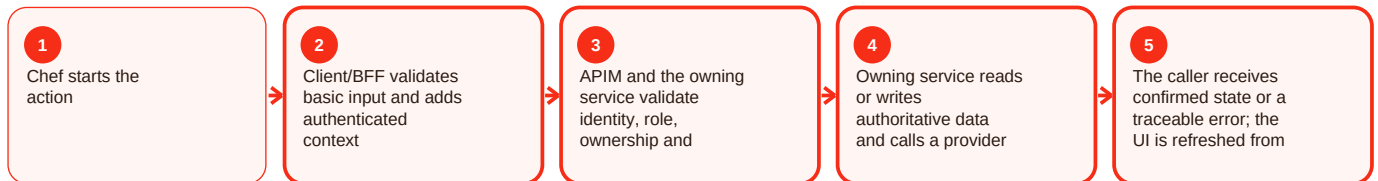
Summary

Customer / chef mode is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Customer/Chef mode, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/user-chef-service/README.md`; `apps/customer-web-next/src/components/chef/`; `apps/customer-web-next/src/components/chef-mode/`.

Chef menu

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

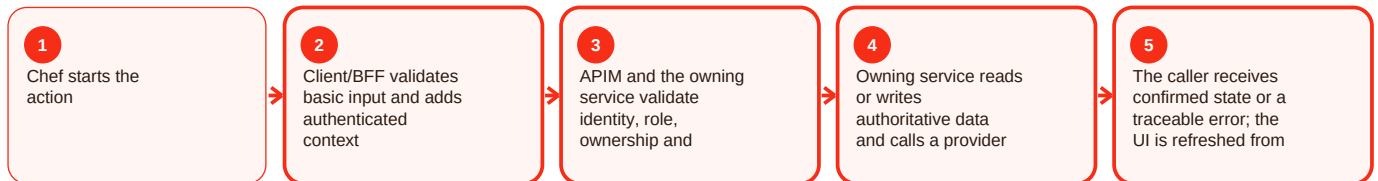
Summary

Chef menu is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Chef menu, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/user-chef-service/README.md`; `apps/customer-web-next/src/components/chef/`; `apps/customer-web-next/src/components/chef-mode/`.

Chef meal creation

Current implementation

Who uses it

Customer

Current status

Current implementation

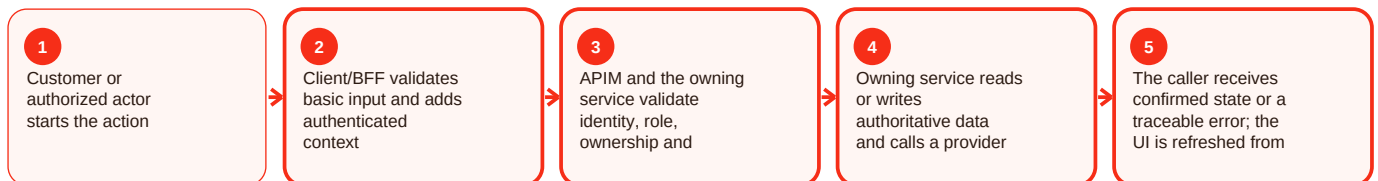
Summary

Chef meal creation is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Chef meal creation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Chef kitchen

Current implementation

Who uses it

Customer

Current status

Current implementation

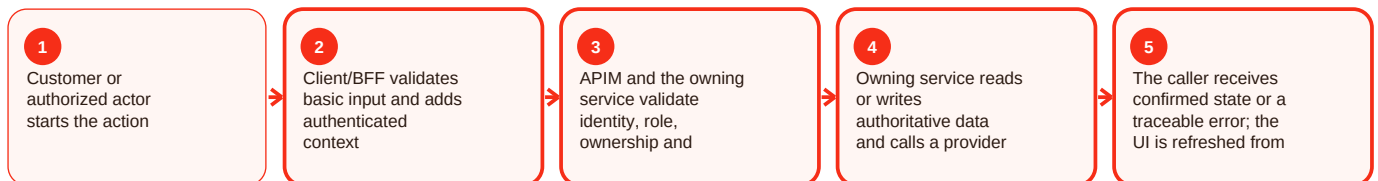
Summary

Chef kitchen is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Chef kitchen, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Chef orders

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

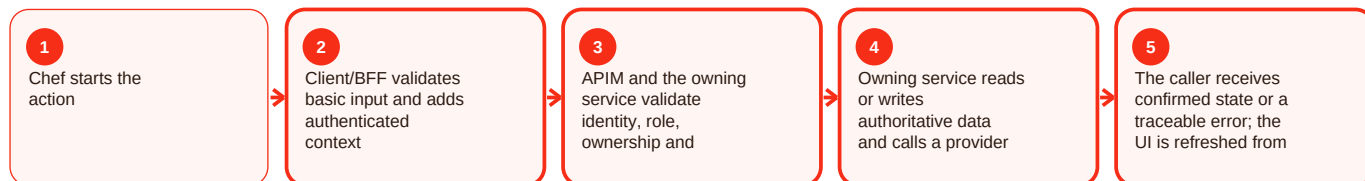
Summary

Chef orders is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Chef orders, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/user-chef-service/README.md`; `apps/customer-web-next/src/components/chef/`; `apps/customer-web-next/src/components/chef-mode/`.

Chef earnings

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

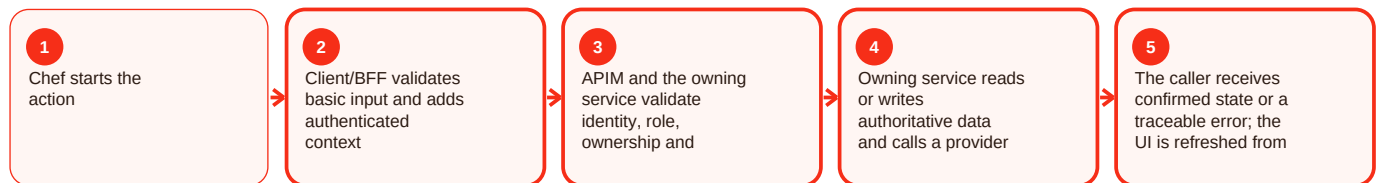
Summary

Chef earnings is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Chef earnings, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: `services/user-chef-service/README.md`; `apps/customer-web-next/src/components/chef/`; `apps/customer-web-next/src/components/chef-mode/`.

Admin shell

Current implementation

Who uses it

Authorized Craves staff

Current status

Current implementation

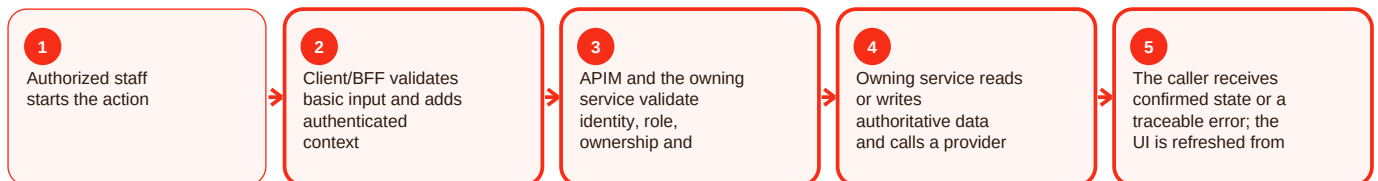
Summary

Admin shell is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For Admin shell, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Admin action denied: Caller lacks admin role/product permission. What to do: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. What to do: Stop, capture audit/request evidence and follow the documented recovery path.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md.

Chef review

Current implementation

Who uses it

Home chef / approved chef

Current status

Current implementation

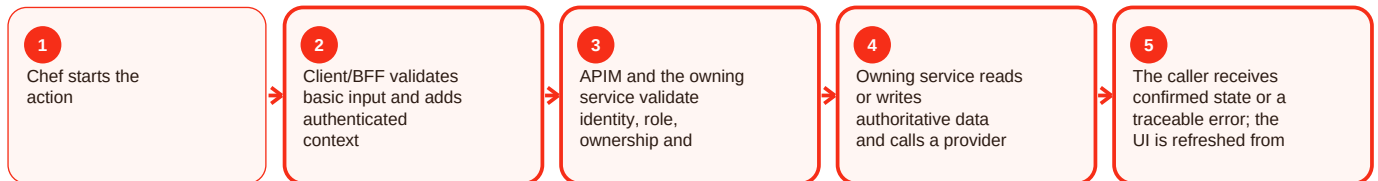
Summary

Chef review is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For Chef review, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/.

Support operations

Current implementation

Who uses it

Authorized Craves staff

Current status

Current implementation

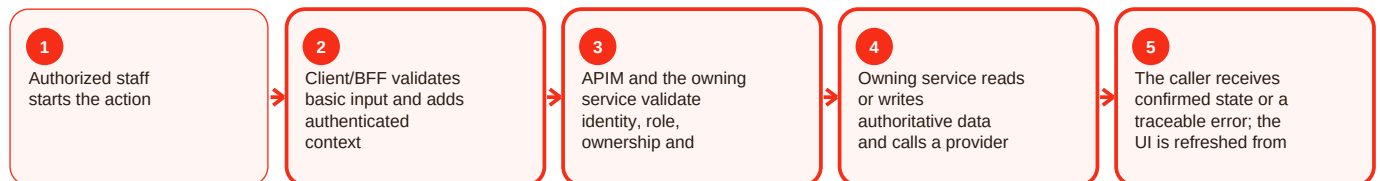
Summary

Support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For Support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Admin action denied: Caller lacks admin role/product permission. What to do: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. What to do: Stop, capture audit/request evidence and follow the documented recovery path.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md.

Staff access

Current implementation

Who uses it

Authorized Craves staff

Current status

Current implementation

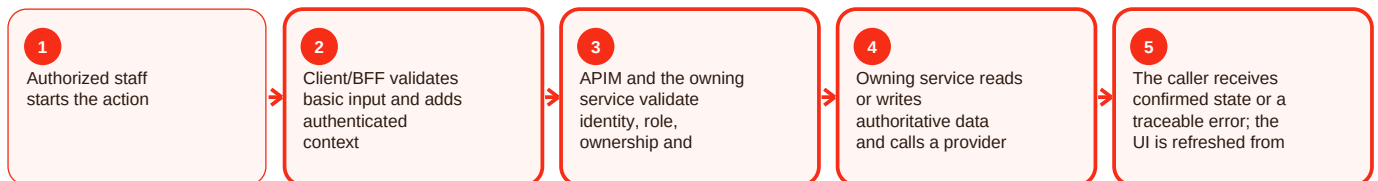
Summary

Staff access is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For Staff access, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Admin action denied: Caller lacks admin role/product permission. What to do: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. What to do: Stop, capture audit/request evidence and follow the documented recovery path.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md.

Catalog administration

Current implementation

Who uses it

Customer

Current status

Current implementation

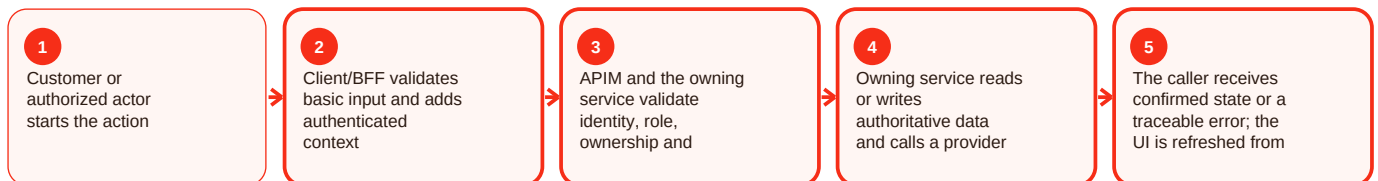
Summary

Catalog administration is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Catalog administration, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Catalog bulk pause/import

Current implementation

Who uses it

Customer

Current status

Current implementation

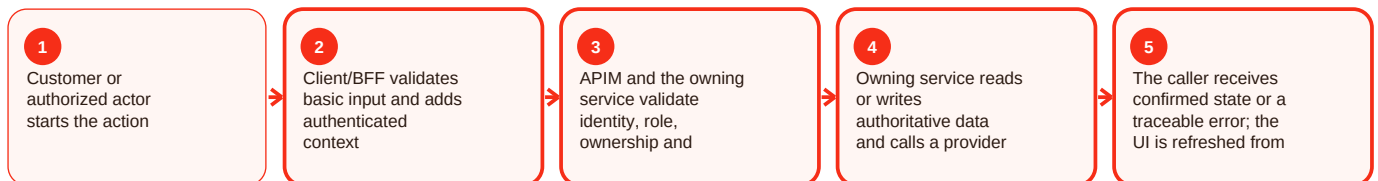
Summary

Catalog bulk pause / import is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For Catalog bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Nearby results empty: No active geocoded kitchen with sellable items in query radius. What to do: Confirm kitchen coordinates, active menu items and caller radius.
Menu item cannot sell: Packaging/availability/status metadata incomplete. What to do: Chef must provide valid required metadata and make the item available.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md.

Promotions

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

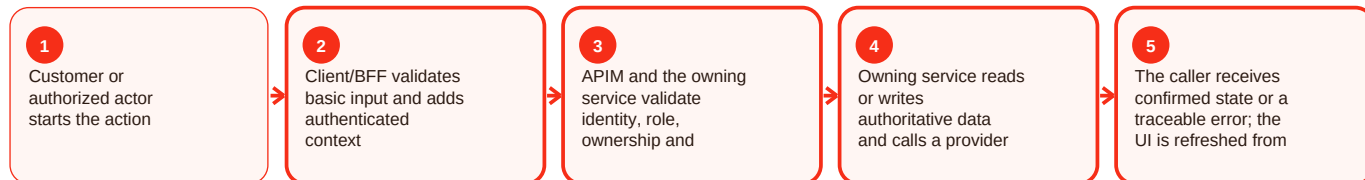
Summary

Promotions is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Promotions, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Community Credit

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

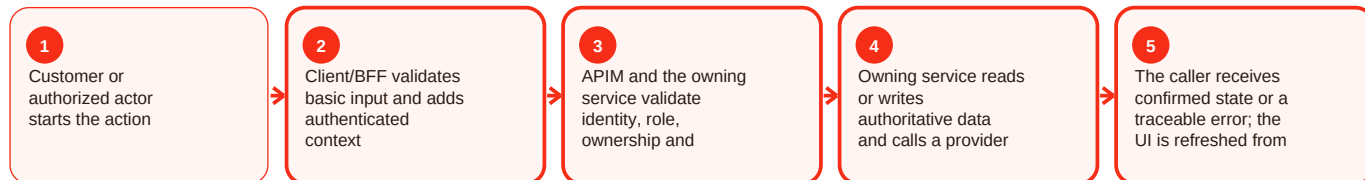
Summary

Community credit is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Community Credit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Consent center

Current implementation

Who uses it

Customer

Current status

Current implementation

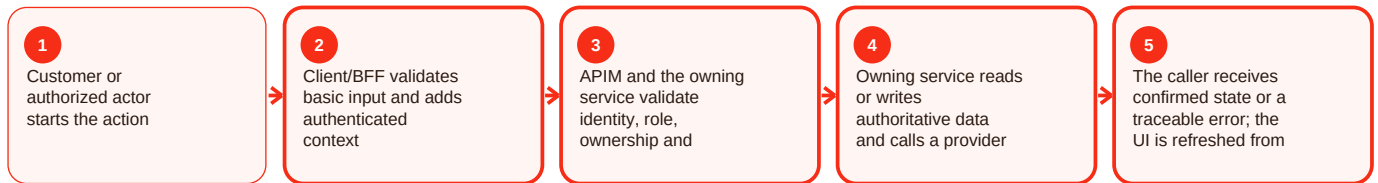
Summary

Consent center is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For Consent center, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/.

Data export

Current implementation

Who uses it

Customer

Current status

Current implementation

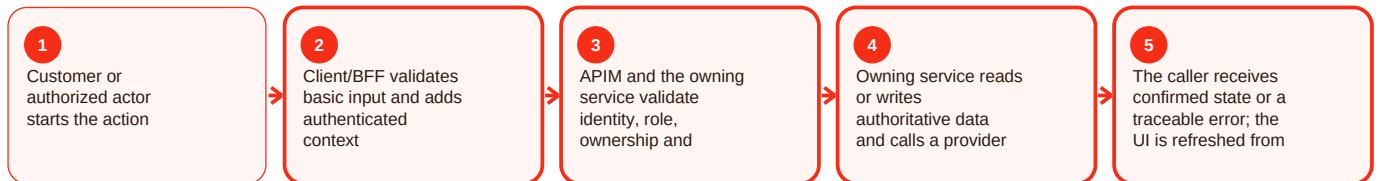
Summary

Data export is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For Data export, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/.

Account deletion

Current implementation

Who uses it

Customer

Current status

Current implementation

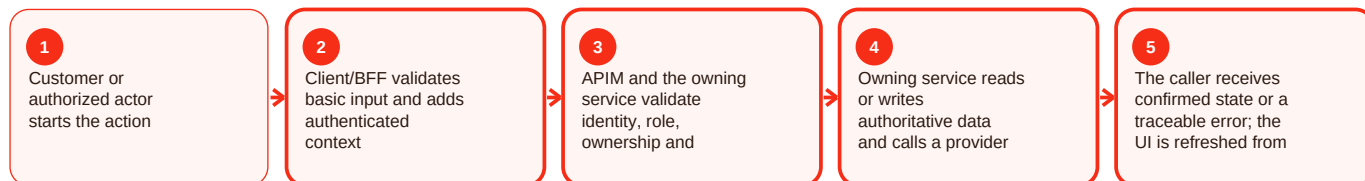
Summary

Account deletion is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For Account deletion, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/.

Catering controls

Current implementation

Who uses it

Customer, chef or staff depending on the feature

Current status

Current implementation

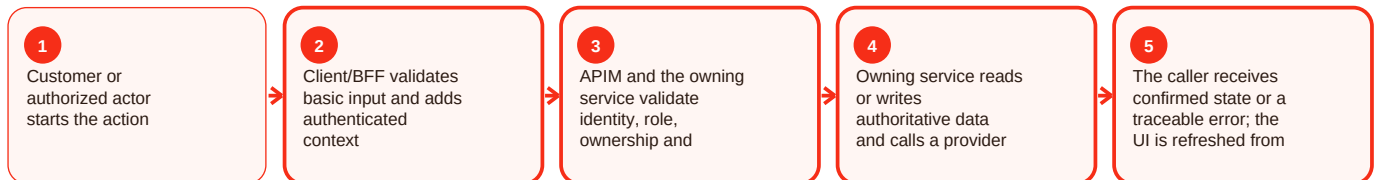
Summary

Catering controls is part of the platform area of Craves. Craves is a two-sided food marketplace connecting customers with home-chef supply. The repository contains a newer Next.js customer/chef web application, seven Spring Boot services, APIM guidance, legacy web/admin/API paths, infrastructure assets and milestone handovers. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The architectural direction is a front-door pattern: clients authenticate, use a same-origin Backend-for-Frontend or APIM, and reach domain services rather than databases directly. Customer and chef journeys share identity while privileged capabilities remain role-gated. For Catering controls, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: README.md; apps/customer-web-next/README.md; services/*/README.md; infra/apim/README.md; docs/handover/.

Catering inquiry/checkout

Current implementation

Who uses it

Customer

Current status

Current implementation

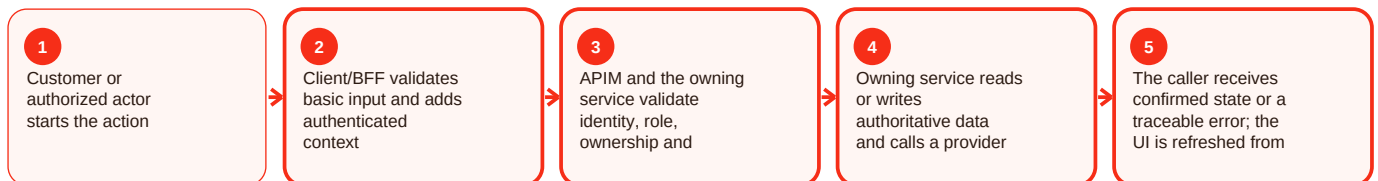
Summary

Catering inquiry / checkout is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For Catering inquiry/checkout, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Checkout rejected: Saved address, cart or catalog state is not valid. What to do: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. What to do: Show the backend quote; do not preserve a client-calculated amount.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md.

AI semantic concierge

Current implementation

Who uses it

Customer

Current status

Current implementation

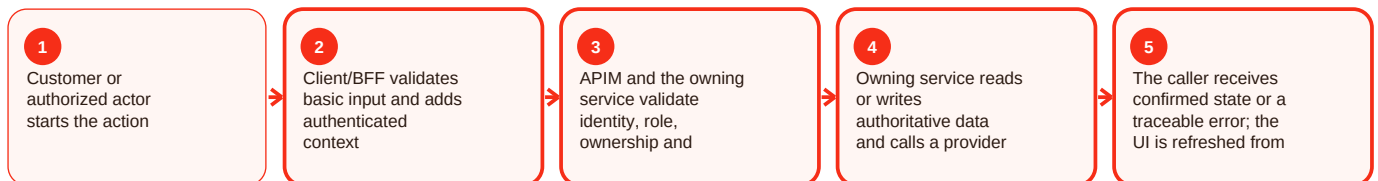
Summary

AI semantic concierge is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For AI semantic concierge, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b.

Google Maps adapter

Current implementation

Who uses it

Customer

Current status

Current implementation

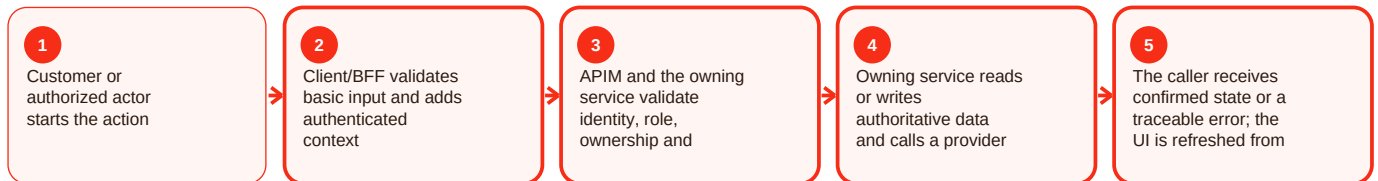
Summary

Google maps adapter is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For Google Maps adapter, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Current location unavailable: Permission/device/geocoder failed. What to do: Allow manual saved-address entry and keep checkout dependent on a valid saved address.
Address rejected at checkout: Address is missing ownership/required fields/coordinates. What to do: Fix the saved address in User-Chef before placing the order.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md.

Google Calendar adapter

Current implementation

Who uses it

Engineering / operations

Current status

Current implementation

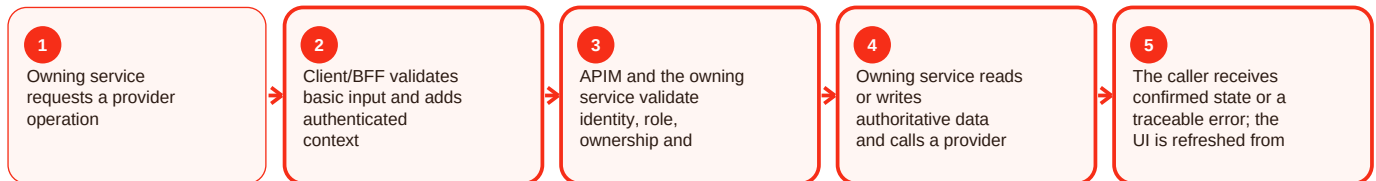
Summary

Google calendar adapter is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For Google Calendar adapter, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Provider is unavailable: External dependency or credential/config problem. What to do:

Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. What to do: Fix and test the adapter rather than leaking vendor logic into domains.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source

evidence: [services/integration-service/README.md](#);

[services/integration-service/**/README.md](#).

Gmail draft adapter

Current implementation

Who uses it

Customer

Current status

Current implementation

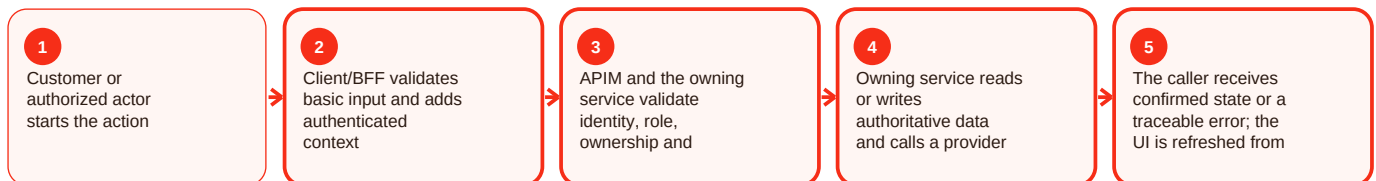
Summary

Gmail draft adapter is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Gmail draft adapter, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

User sees stale state: Client cache/local state differs from backend. What to do: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. What to do: Preserve domain consistency and retry only when safe.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b.

Order multi-database routing

Current implementation

Who uses it

Customer

Current status

Current implementation

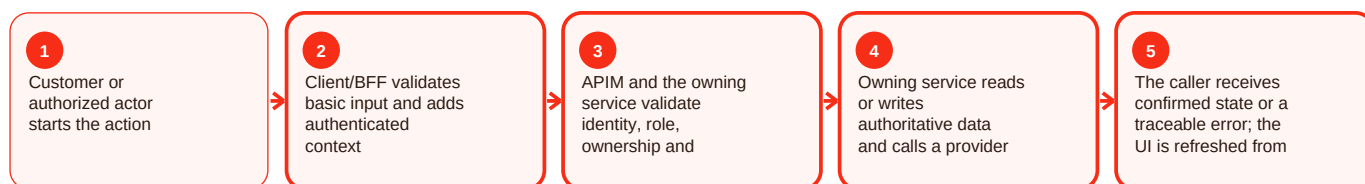
Summary

Order multi-database routing is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

How it works

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For Order multi-database routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

End-to-end flow



Edge cases and support

Order not found: Wrong ID or ownership mismatch. What to do: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. What to do: Reload order state and use only the supported next action.

Evidence

Direct repository evidence supports this capability or architecture boundary. Source evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.

Repository gaps that must stay visible

- APIM README cites `infra/apim/craves-openapi.yaml`, but that literal artifact was not resolvable on main during this evidence snapshot.
- `customer-web-next` README names `azure-pipelines-customer-web-next.yml`, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.